SCALER

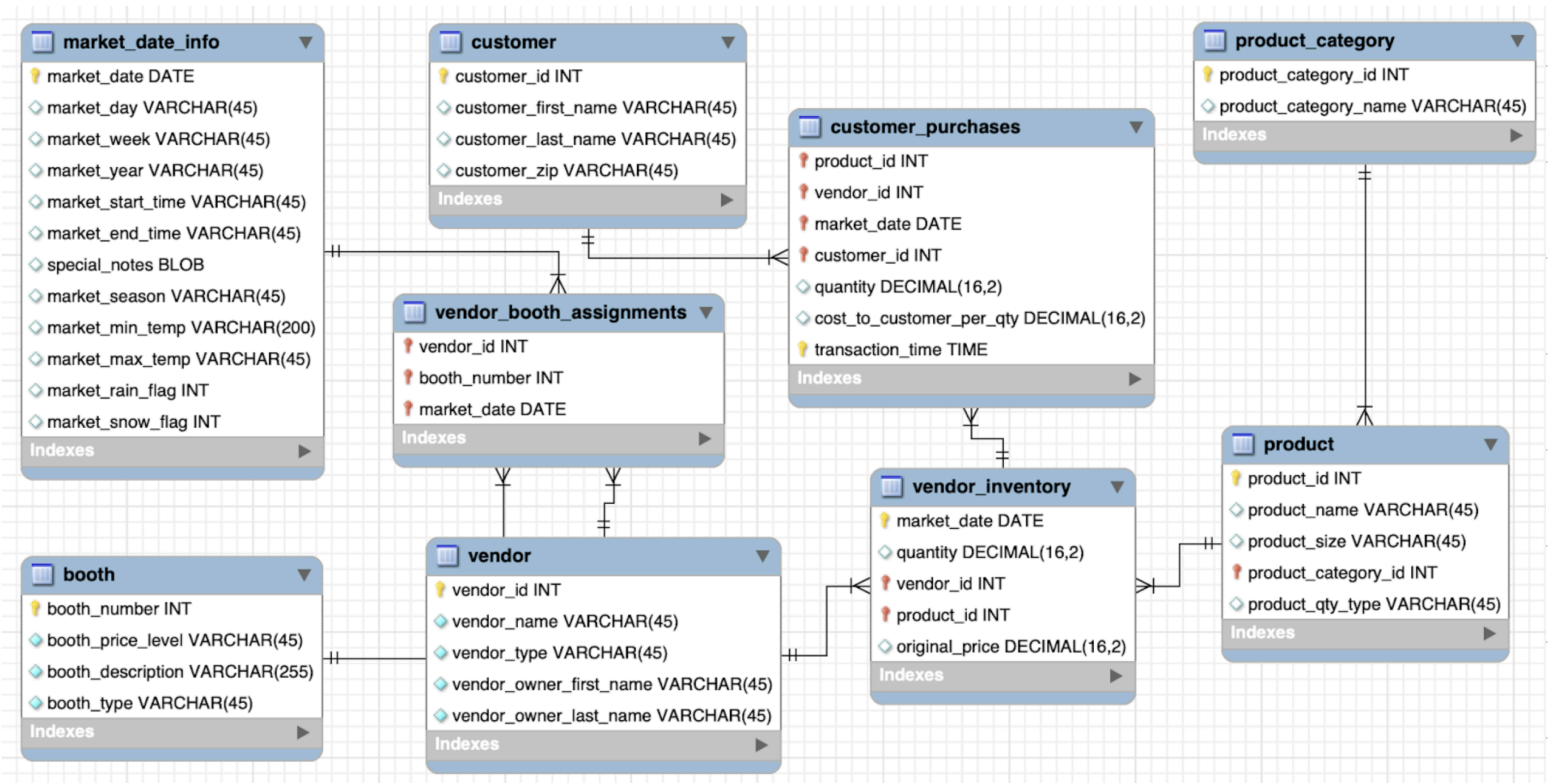
KEEP MOVING

FORWARD

WhatsApp Link

https://chat.whatsapp.com/EvVhJ555AGW1cJaeKI0USe

Dataset



Agenda

1. Window frames
2. MySQL → Big Query
3. Arrays & structs → 5 minutes
4. Index & Partitioning

Window Frames

ID	name	Salary	c-salary
1	Jim	900	900
2	Pam	700	1600
3	Dwight	900	2500
4	Andy	800	3300
5	Kevin	700	4000

→ why do we get running sum instead of total sum

Query

```
select
    id, name, salary
    sum (salary)  over (order by id) as c-salary
from employee ;
```

whenever you have order by in your window functio
a default query → reason for cummulative sum

Range between unbounded preceding and
current row

Terminologies

- unbounded → limiters
- range / rows
- current row (active row)
- preceding (previous rows)
- following (next rows)

ID	name	Salary	C-salary	preceding	CR
1	Jim	900	900	null	900
2	Pam	700	1600	null, 900	700
3	Dwight	900	2500	null, 900, 700	900
4	Andy	800	3200	null, 900, 700, 900	800
5	Kevin	700	4000	null, 900, 700 900, 800	700

order by id

Default Query

- whenever you have an order by clause in window function, the compiler automatically adds a default query

RANGE between unbounded preceding AND current row

Row

Range vs Row

In case of duplicates, range will treat them as same, whereas rows will treat them as separate entity

→ order by firstname in salary

ID	name	Salary
1	Andy	900
2	Dwight	700
3	Jim	900
4	Jim	800
5	Kevin	700
6	Pam	600

duplicate value

Range	Row
900	900
1600	1600
900+700 900+800	2500
900+700 900+800	2500+800
3300 +700	3300 +700
4000+ 600	4000+ 600

Query

```
select
    id, name, salary
    sum (salary)
    over (order by id  Range / Row  between unbounded
    preceding AND current row ) as c-salary
from employee ;
```

Lead and Lag

Lead → next entries

Lag → previous entries

Syntax

Lead (column , row_shift)

row_shift → +ve number, on how many rows to move

column → which column to select.

Question

① Select next heighest and previous salary for each employee

Lead (salary , 1) over (order by salary)

Lag (salary, 1) over (order by salary)

From MySQL to Big Query

→ Majority of the organization, use MySQL as an operational database for real-time metadata, user settings and recent transaction data

1 Year data @ bsense → Hadoop / s3 parquet

Index and partitions

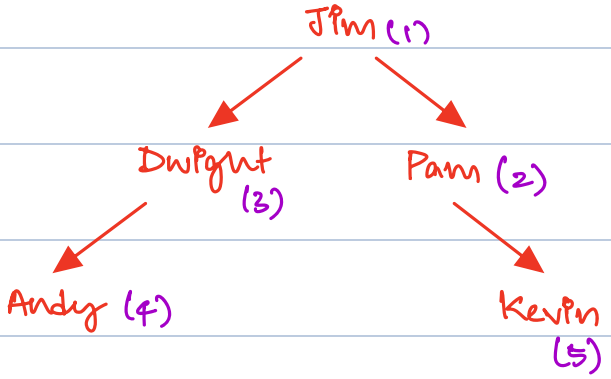
MySQL Index → B-Tree structure

Big Query Index → Search Index

I am creating index on top of name

Id	name	Salary
1	Jim	900
2	Pam	700
3	Dwight	900
4	Andy	800
5	Kevin	700

Ascending order B-Tree



Without Index

Searching for Andy

With Index

Searching for Andy

proper Indexes → 100% help your reads

→ writes will be slower

SQL Example

① Create index on first_name

② Explain without using index, with using index

③ Drop index on first_name

Note:

With index, lookups are faster however the writes become slower.

Big Query - Search Index

① Create search index idx on project.dataset.table;

name	category	title
Michael phelps	Swimming	Gold
Usain Bolt	Running	Gold
Michael Jordan	Basketball	Gold

Text Extraction
↓
Token processing
↓
Inverted index
+
Bloom filters

Inverted Index

"michael" → [1, 3]

"usain" → [2]

"gold" → [1, 2, 3]

② Drop search index idx on project.dataset.table;

Example in Big Query

Partitions - Big Query

① Partitions is a database process to divide very large tables are divided into multiple smaller, individual parts

② Scan lesser data

③ Reduces query execution time

Types of Partitions in Big Query

Time-unit column

→ use this partition when your table has a Date, Datetime, or timestamp column

customer_purchase_DT

Ingestion time

→ use this partition when you dont have reliable timestamp column

log_insertion_time

Integer Range

→ use this partition when your queries often filter by numerical field

census age

Doubts

1. Always better to read lesser data than more data
2. proper indexes rather than over index
3. Cache warming is important

platforms

leetcode ⟶ SQL

hackerrank ⟶ SQL

Drive ⟶ Interview

Non mandatory ⟶ Scaler

Monday

---

1. strong suites
2. learn newer concepts using visualizations